PYTHON

ACTIVITY BASED LEARNING

CASE STUDY 1: UNIVERSITY SCHOLARSHIP ALLOCATION

1. Problem Statement

Evaluate applicants using aggregate marks, subject minimums, attendance, and family income. Rank eligible applicants and break ties using attendance.

2. Objectives

- Apply the required rules to the supplied data.
- Use beginner-level Python constructs from the assigned syllabus.
- Display a clear and understandable result.

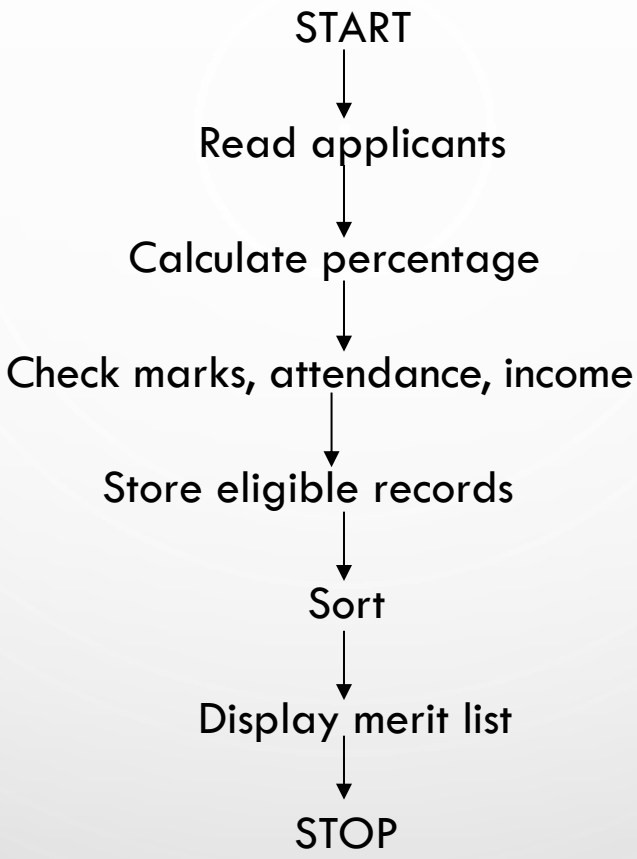
3. Assumptions

- Three subjects, each out of 100; aggregate at least 75%; each subject at least 60; attendance at least 85%; annual income at most Rs. 300000.
- If percentages tie, higher attendance ranks first. These are illustrative rules.

4. Algorithm

1. Read applicant records.
2. Calculate percentage = total marks / 3.
3. Check every eligibility condition.
4. Store eligible applicants as tuples (name, percentage, attendance).
5. Sort by percentage and then attendance in descending order.
6. Display the merit list and rejected applicants.

5. Flowchart



6. Pseudocode

```
INPUT applicants
eligible = empty list
FOR each applicant
        percentage = sum(marks) / 3
        IF percentage >= 75
        AND minimum(marks) >= 60
```

AND attendance >= 85 AND income <= 300000 ADD (name, percentage, attendance) to eligible

ELSE DISPLAY not eligible

SORT eligible by percentage descending, attendance descending

DISPLAY eligible

STOP

7. Python Source Code

```
applicants = [
        ("Anu", [85, 90, 80], 92, 250000),
        ("Balu", [78, 82, 80], 88, 280000),
        ("Chitra", [95, 55, 90], 96, 200000),
        ("Deepak", [88, 86, 84], 84, 220000)
 ]
eligible = [ ]
for name, marks, attendance, income in applicants:
        percentage = sum(marks) / 3
        if percentage >= 75 and min(marks) >= 60 and attendance >= 85 and income <= 300000:
                eligible.append((name, percentage, attendance))
        else:
                print(name, "is not eligible")
eligible.sort(key=lambda x: (x[1], x[2]), reverse=True)
print("SCHOLARSHIP MERIT LIST")
for rank, record in enumerate(eligible, 1):
        print(rank, record[0], round(record[1], 2), "Attendance:", record[2])
```

8. Test Cases

Tests:

Anu [85,90,80],

attendance 92,

income 250000 ⟶ eligible,85%.

Chitra [95,55,90] ⟶ rejected (subject minimum).

Deepak attendance 84 ⟶ rejected.

9. Sample Output

Output:

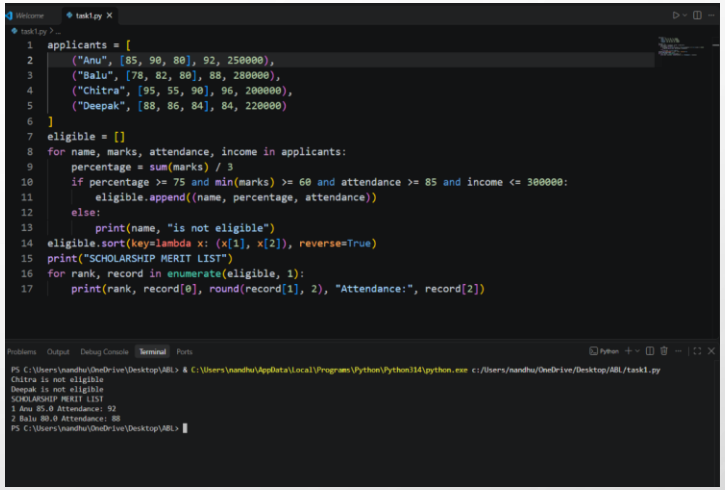
Chitra is not eligible

Deepak is not eligible

SCHOLARSHIP MERIT LIST

1 Anu 85.0 Attendance: 92

2 Balu 80.0 Attendance: 88



10. Conclusion

The program checks multiple eligibility rules and ranks qualified applicants. It uses lists, tuples, loops, conditions, sorting, and arithmetic.

CASE STUDY 2: PROGRESSIVE ELECTRICITY BILLING

1. Problem Statement

Calculate household bills using consumption slabs, fixed charges, surcharge, and rebate; compare bills across months and flag unusual increases.

2. Objectives

- Apply the required rules to the supplied data.
- Use beginner-level Python constructs from the assigned syllabus.
- Display a clear and understandable result.

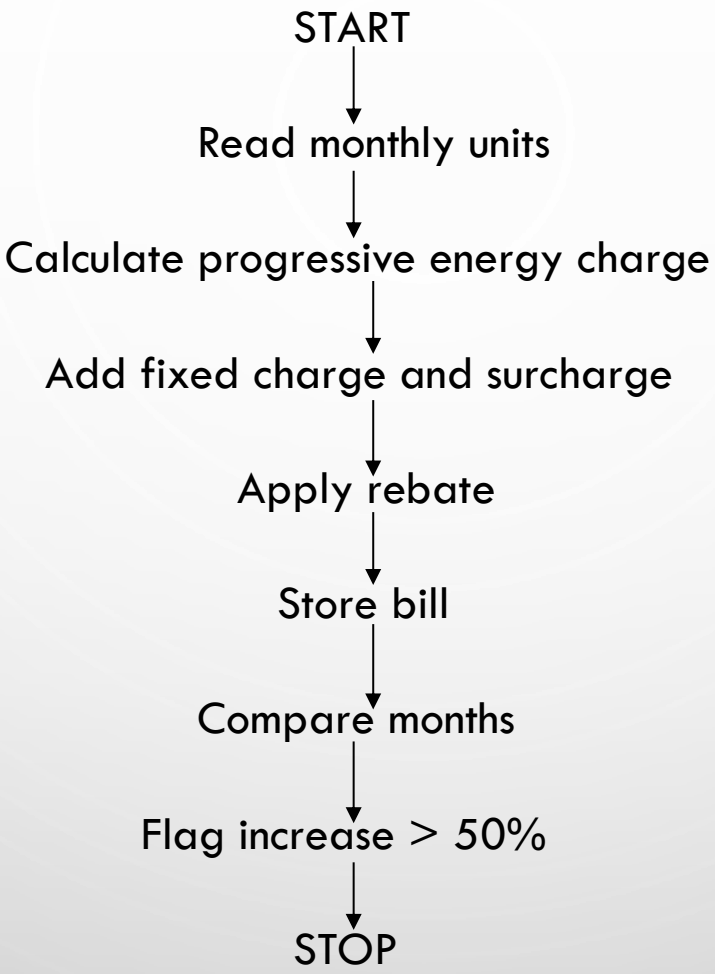
3. Assumptions

- First 100 units: Rs. 2/unit; next 100: Rs. 3/unit; above 200: Rs. 5/unit.
- Fixed charge Rs. 50; surcharge 5% of energy charge; rebate Rs. 20 for consumption at most 100 units.
- Increase above 50% from previous month is unusual. All tariff values are illustrative.

4. Algorithm

1. Read monthly consumption values.
2. Calculate energy charge slab by slab.
3. Add fixed charge and surcharge.
4. Apply rebate for low consumption.
5. Store each bill.
6. Compare each month with previous month and flag increases above 50%.

5. Flowchart



6. Pseudocode

```
FOR each month's units
    IF units <= 100, energy = units * 2
    ELSE IF units <= 200, energy = 100*2 + (units-100)*3
```

```
        ELSE energy = 100*2 + 100*3 + (units-200)*5
        bill = energy + 50 + energy*0.05
        IF units <= 100, bill = bill - 20
        STORE bill
FOR each bill after first
        increase = (current - previous) / previous * 100
        IF increase > 50, DISPLAY unusual
DISPLAY monthly bills
STOP
```

7. Python Source Code

```
monthly_units = [80, 120, 260]
bills = [ ]
for units in monthly_units:
        if units <= 100:
                energy = units * 2
        elif units <= 200:
                energy = 100 * 2 + (units - 100) * 3
        else:
                energy = 100 * 2 + 100 * 3 + (units - 200) * 5
        bill = energy + 50 + energy * 0.05
        if units <= 100:
                bill -= 20
        bills.append(bill) detected") else: print("No unusual increase")
```

```
print("MONTHLY ELECTRICITY BILLS")
for i in range(len(bills)):
    print("Month", i + 1, "Units:", monthly units[i],
        "Bill: Rs.", round(bills[i], 2))
    print("MONTH-TO-MONTH COMPARISON")
    for i in range(1, len(bills)):
        if bills[i - 1] > 0:
            increase = (bills[i] - bills[i - 1]) / bills[i - 1] * 100
            print("Month", i + 1, "increase:", round(increase, 2), "%")
            if increase > 50:
                print("Unusual increase detected")
            else:
                print("No unusual increase")
```

8. Test Cases

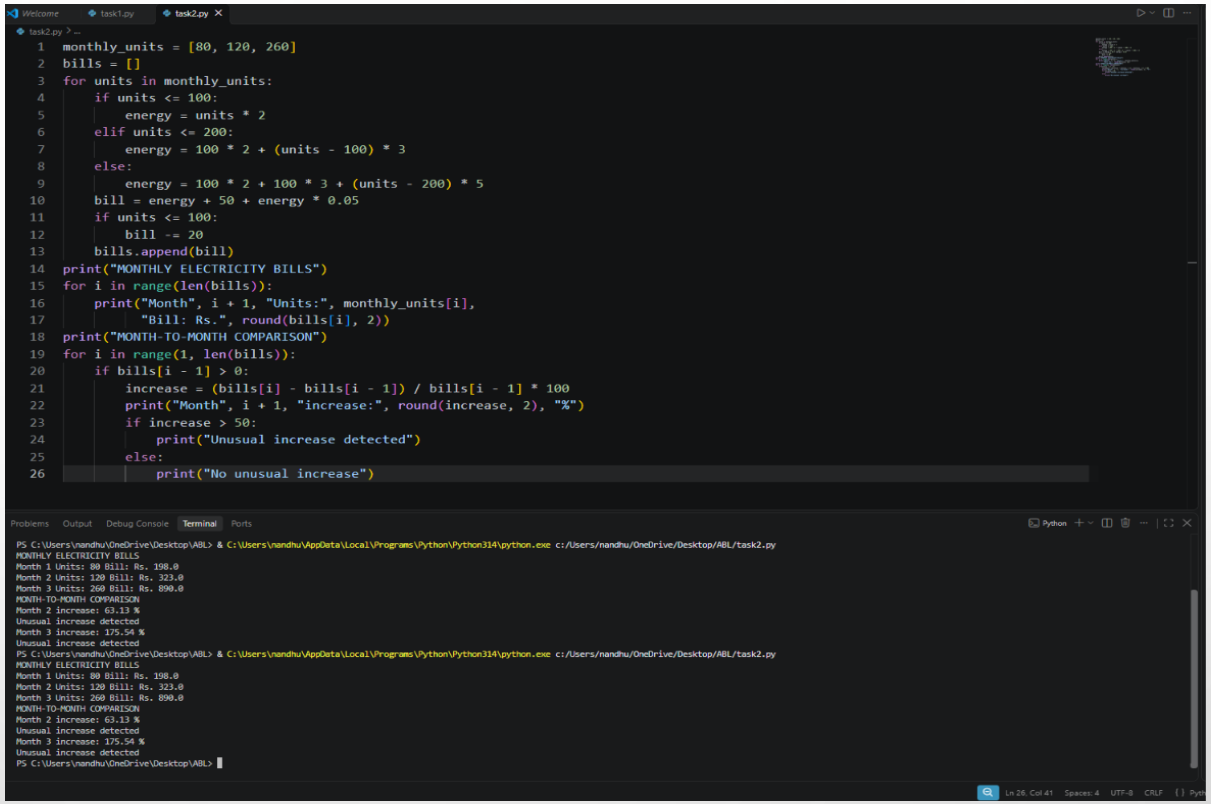
Tests:

80 units ⟶ energy 160,
surcharge 8,
fixed 50,
rebate 20,
bill Rs. 198. 120 units ⟶ bill Rs. 323. 260 units ⟶ bill Rs. 785.

9. Sample Output

Output:

MONTHLY ELECTRICITY BILLS
Month 1 Units: 80 Bill: Rs. 198.0
Month 2 Units: 120 Bill: Rs. 323.0
Month 3 Units: 260 Bill: Rs. 785.0
MONTH-TO-MONTH COMPARISON
Month 2 increase: 63.13 %
Unusual increase detected
Month 3 increase: 142.72 %
Unusual increase detected



```python
monthly_units = [80, 120, 260]
bills = []
for units in monthly_units:
    if units <= 100:
        energy = units * 2
    elif units <= 200:
        energy = 100 * 2 + (units - 100) * 3
    else:
        energy = 100 * 2 + 100 * 3 + (units - 200) * 5
    bill = energy + 50 + energy * 0.05
    if units <= 100:
        bill -= 20
    bills.append(bill)
print("MONTHLY ELECTRICITY BILLS")
for i in range(len(bills)):
    print("Month", i + 1, "Units:", monthly_units[i],
          "Bill: Rs.", round(bills[i], 2))
print("MONTH-TO-MONTH COMPARISON")
for i in range(1, len(bills)):
    if bills[i - 1] > 0:
        increase = (bills[i] - bills[i - 1]) / bills[i - 1] * 100
        print("Month", i + 1, "increase:", round(increase, 2), "%")
        if increase > 50:
            print("Unusual increase detected")
        else:
            print("No unusual increase")
```

```
PS C:\Users\nandhu\OneDrive\Desktop\ABL> & C:\Users\nandhu\AppData\Local\Programs\Python\Python314\python.exe c:/Users/nandhu/OneDrive/Desktop/ABL/task2.py
MONTHLY ELECTRICITY BILLS
Month 1 Units: 80 Bill: Rs. 198.0
Month 2 Units: 120 Bill: Rs. 323.0
Month 3 Units: 260 Bill: Rs. 890.0
MONTH-TO-MONTH COMPARISON
Month 2 increase: 63.13 %
Unusual increase detected
Month 3 increase: 175.54 %
Unusual increase detected
PS C:\Users\nandhu\OneDrive\Desktop\ABL> & C:\Users\nandhu\AppData\Local\Programs\Python\Python314\python.exe c:/Users/nandhu/OneDrive/Desktop/ABL/task2.py
MONTHLY ELECTRICITY BILLS
Month 1 Units: 80 Bill: Rs. 198.0
Month 2 Units: 120 Bill: Rs. 323.0
Month 3 Units: 260 Bill: Rs. 890.0
MONTH-TO-MONTH COMPARISON
Month 2 increase: 63.13 %
Unusual increase detected
Month 3 increase: 175.54 %
Unusual increase detected
PS C:\Users\nandhu\OneDrive\Desktop\ABL>
```

10. Conclusion

The program calculates slab-based bills and compares monthly increases using conditions, loops, lists, and arithmetic.

CASE STUDY 3: RETAIL BILLING WITH MULTIPLE DISCOUNT RULES

1. Problem Statement

Create an itemized retail bill using quantities, category-specific tax rates, a threshold discount, and a maximum concession.

2. Objectives

- Apply the required rules to the supplied data.
- Use beginner-level Python constructs from the assigned syllabus.
- Display a clear and understandable result.

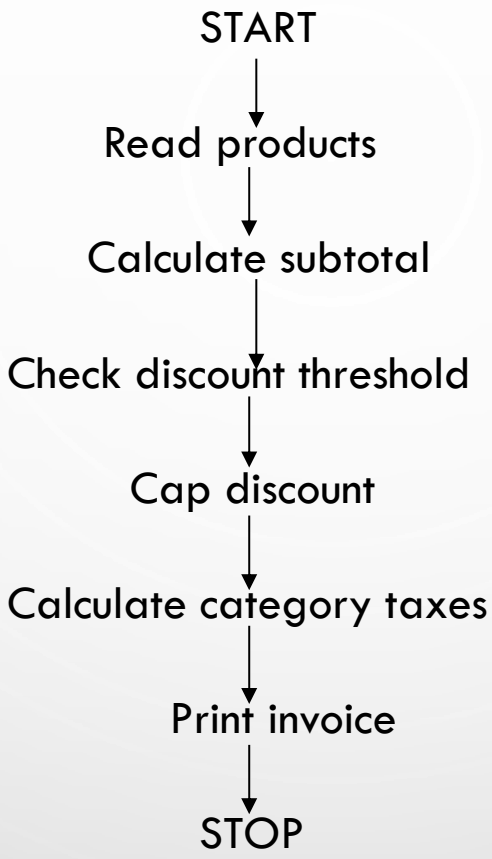
3. Assumptions

- Discount is 10% when subtotal is at least Rs. 2000, capped at Rs. 500.
- Tax: electronics 18%, clothing 5%, other 12%. Discount is distributed proportionately before tax. Values are illustrative.

4. Algorithm

1. Read product, quantity, price, and category records.
2. Calculate each item amount and subtotal.
3. Apply threshold discount and maximum limit.
4. Find the discount proportion for each item.
5. Calculate category tax on discounted item amounts.
6. Display itemized bill and final total.

5. Flowchart



6. Pseudocode

```
subtotal = sum(price * quantity)
IF subtotal >= 2000
        discount = minimum(subtotal * 0.10, 500)
ELSE
discount = 0 discount_rate = discount / subtotal
FOR each product
        discounted_amount = item_amount * (1 - discount_rate)
```

```
        SELECT tax rate using category
        ADD discounted_amount * tax_rate to total_tax
final_bill = subtotal - discount + total_tax
DISPLAY invoice
STOP
```

7. Python Source Code

```
products = [
        ("Notebook", 5, 60, "other"),
        ("Shirt", 2, 500, "clothing"),
        ("Headphones", 1, 1800, "electronics")
]
subtotal = 0
for name, qty, price, category in products:
        subtotal += qty * price
discount = 0
if subtotal >= 2000:
        discount = min(subtotal * 0.10, 500)
rate = discount / subtotal if subtotal > 0 else 0
total_tax = 0
print("ITEMIZED BILL")
for name, qty, price, category in products:
        amount = qty * price * (1 - rate)
        if category == "electronics":
                tax_rate = 0.18
```

```
        elif category == "clothing":
                tax_rate = 0.05
        else:
                tax_rate = 0.12
        tax = amount * tax_rate
        total_tax += tax
        print(name, "Qty:", qty, "Tax:", round(tax, 2))
print("Subtotal:", subtotal)
print("Discount:", round(discount, 2))
print("Tax:", round(total_tax, 2))
print("Final bill:", round(subtotal - discount + total_tax, 2))
```

8. Test Cases

Tests:

Subtotal Rs. 800 ⟶ no discount.

Subtotal Rs. 3000 ⟶ discount Rs. 300.

Subtotal Rs. 8000 ⟶ discount capped at Rs. 500.

9. Sample Output

Output:

ITEMIZED BILL

Notebook Qty: 5 Tax: 32.73

Shirt Qty: 2 Tax: 45.45

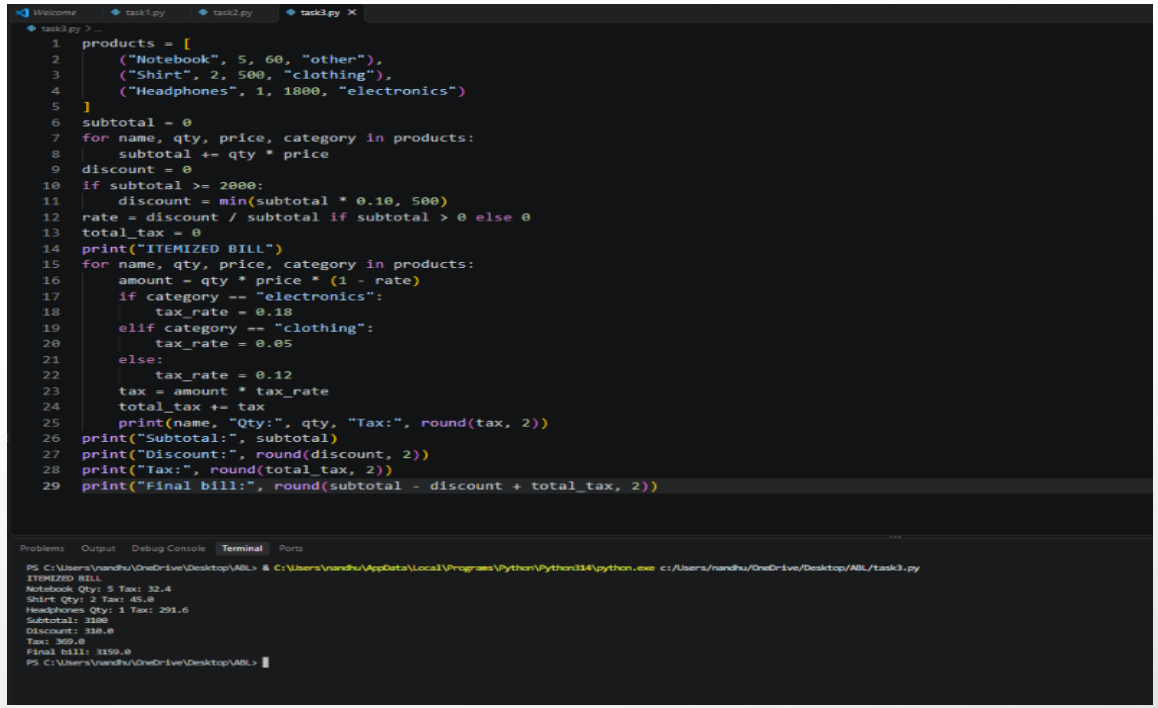
Headphones Qty: 1 Tax: 294.55

Subtotal: 3100

Discount: 310.0

Tax: 372.73

Final bill: 3162.73



```python
products = [
    ("Notebook", 5, 60, "other"),
    ("Shirt", 2, 500, "clothing"),
    ("Headphones", 1, 1800, "electronics")
]
subtotal = 0
for name, qty, price, category in products:
    subtotal += qty * price
discount = 0
if subtotal >= 2000:
    discount = min(subtotal * 0.10, 500)
rate = discount / subtotal if subtotal > 0 else 0
total_tax = 0
print("ITEMIZED BILL")
for name, qty, price, category in products:
    amount = qty * price * (1 - rate)
    if category == "electronics":
        tax_rate = 0.18
    elif category == "clothing":
        tax_rate = 0.05
    else:
        tax_rate = 0.12
    tax = amount * tax_rate
    total_tax += tax
    print(name, "Qty:", qty, "Tax:", round(tax, 2))
print("Subtotal:", subtotal)
print("Discount:", round(discount, 2))
print("Tax:", round(total_tax, 2))
print("Final bill:", round(subtotal - discount + total_tax, 2))
```

```
PS C:\Users\nandhu\OneDrive\Desktop\ABL> & C:\Users\nandhu\AppData\Local\Programs\Python\Python314\python.exe c:/Users/nandhu/OneDrive/Desktop/ABL/task3.py
ITEMIZED BILL
Notebook Qty: 5 Tax: 32.4
Shirt Qty: 2 Tax: 45.0
Headphones Qty: 1 Tax: 291.6
Subtotal: 3100
Discount: 310.0
Tax: 369.0
Final bill: 3159.0
PS C:\Users\nandhu\OneDrive\Desktop\ABL>
```

10. Conclusion

The program prepares an invoice with discounts and category-based taxes using lists, tuples, loops, conditions, and arithmetic.

CASE STUDY 4: EXAMINATION RESULT AND GRACE-MARK VALIDATION

1. Problem Statement

Process marks in multiple subjects, apply a limited grace-mark policy, and generate the final result without exceeding maximum marks.

2. Objectives

- Apply the required rules to the supplied data.
- Use beginner-level Python constructs from the assigned syllabus.
- Display a clear and understandable result.

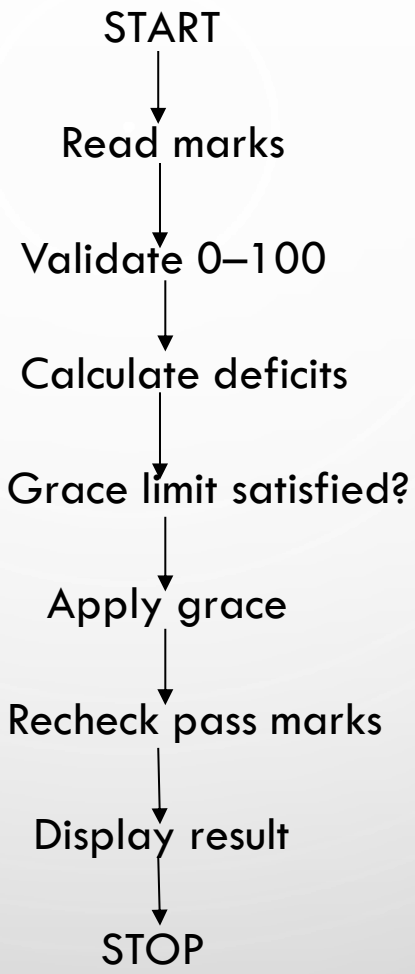
3. Assumptions

- Four subjects, each out of 100; pass mark is 40.
- Maximum total grace is 5 marks. Grace can cover subject deficits of 1–5 marks; each deficient subject is raised to 40 only if total deficit is at most 5.

4. Algorithm

1. Read and validate marks.
2. Find marks below 40 and calculate deficits.
3. If total deficit is within 5, add the required grace marks.
4. Check all subjects again.
5. Display final marks and PASS or FAIL.

5. Flowchart



6. Pseudocode

INPUT marks
IF any mark < 0 OR > 100, DISPLAY invalid and STOP
deficits = pass_mark - each mark below 40
IF sum(deficits) <= 5 AND each deficit <= 5

ADD each deficit to its subject
IF every final mark >= 40
DISPLAY PASS
ELSE DISPLAY FAIL
DISPLAY final marks
STOP

7. Python Source Code

```
marks = [38, 45, 39, 72]
pass_mark = 40
valid = True
for mark in marks:
    if mark < 0 or mark > 100:
        valid = False
if not valid:
    print("Invalid marks")
else:
    deficits = [ ]
    for mark in marks:
        if mark < pass_mark:
            deficits.append(pass_mark - mark)
    if sum(deficits) <= 5 and max(deficits + [0]) <= 5:
        for i in range(len(marks)):
        if marks[i] < pass_mark:
            marks[i] = pass_mark
```

```
            print("Grace marks applied:", sum(deficits))
        else:
            print("Grace policy cannot cover shortage")
    print("Final marks:", marks)
    passed = True
    for mark in marks:
        if mark < pass_mark:
            passed = False
    if passed:
        print("Result: PASS")
    else:
        print("Result: FAIL")
```

8. Test Cases

Tests:

[38,45,39,72]⟶deficits 2+1;

grace 3;

PASS. [30,45,39,72]⟶deficit 10;

grace insufficient;

FAIL. [40,55,60,70]⟶ no grace needed;

PASS.

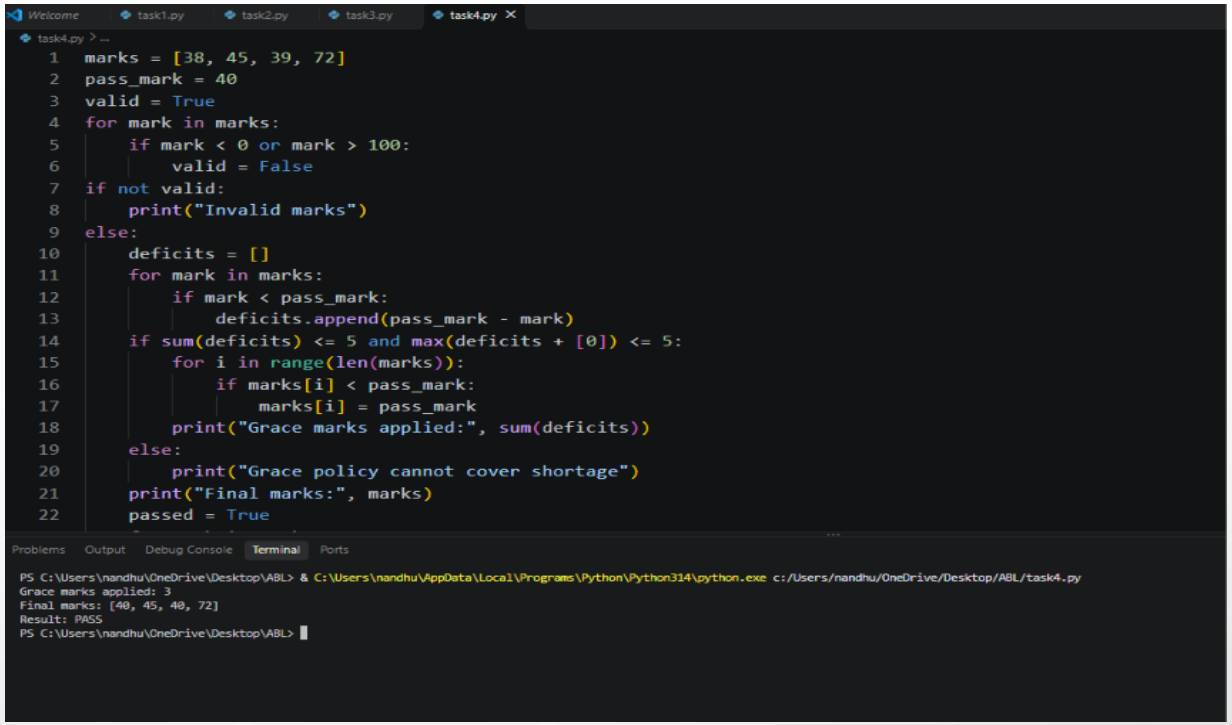
9. Sample Output

Output:

Grace marks applied: 3

Final marks: [40, 45, 40, 72]

Result: PASS



10. Conclusion

The program validates marks, applies a limited grace policy, and checks the final subject-wise result.

CASE STUDY 5: ATTENDANCE RECOVERY COMPUTATION

1. Problem Statement

Calculate current attendance and find the minimum future classes needed to reach an eligibility percentage under different attendance scenarios.

2. Objectives

- Apply the required rules to the supplied data.
- Use beginner-level Python constructs from the assigned syllabus.
- Display a clear and understandable result.

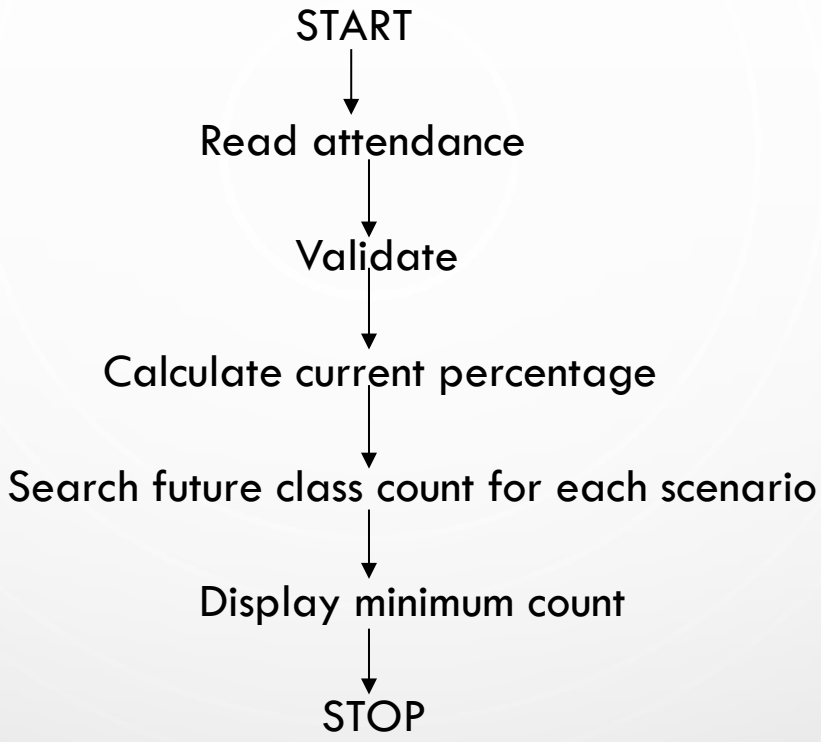
3. Assumptions

- Sample input: 60 attended out of 90 conducted; target is 75%.
- Scenario A: attend every future class. Scenario B: attend 4 out of each group of 5 future classes.

4. Algorithm

1. Calculate current percentage.
2. Use a loop to find minimum future classes if all are attended.
3. Test future class counts for the 4-out-of-5 scenario.
4. Display the required class counts.

5. Flowchart



6. Pseudocode

```
attended = 60; conducted = 90; target = 75
current = attended / conducted * 100
count = 0
WHILE (attended + count) / (conducted + count) * 100 < target
        count = count + 1
DISPLAY count for all-attend scenario
future = 1
```

WHILE target not reached
    future_attended = (future // 5) * 4 + min(future % 5, 4)
    calculate new percentage
    IF target reached, DISPLAY future
    ELSE future = future + 1
STOP

7. Python Source Code

```
attended = 60
conducted = 90
target = 75
if conducted <= 0 or attended < 0 or attended > conducted:
    print("Invalid attendance")
else:
    print("Current attendance:", round(attended / conducted * 100, 2), "%")
    count = 0
    while (attended + count) / (conducted + count) * 100 < target:
        count += 1
    print("Future classes if all are attended:", count)
    future = 1
    while True:
        future_attended = (future // 5) * 4 + min(future % 5, 4)
        percentage = (attended + future_attended) / (conducted + future) * 100
        if percentage >= target:
```

```
            break
        future += 1
print("Future classes in 4-out-of-5 scenario:", future)
```

8. Test Cases

Tests:

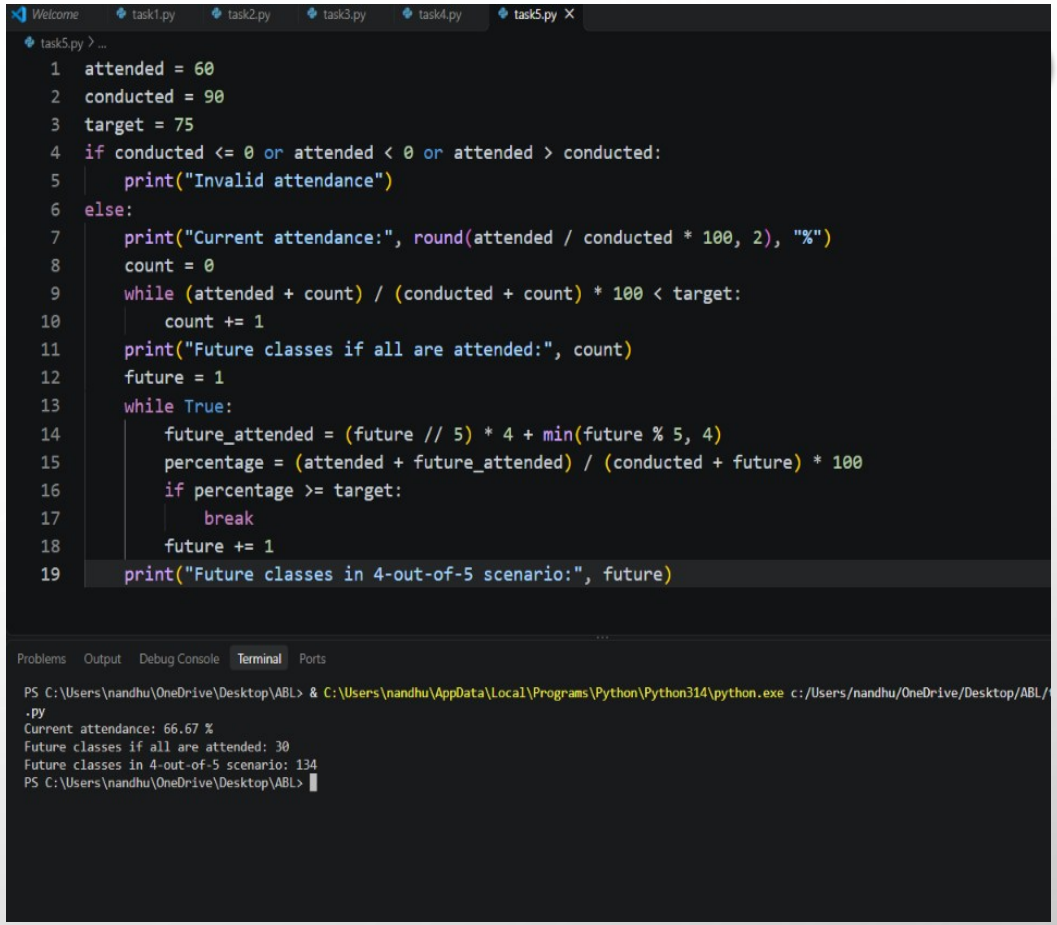
60/90,

target 75%,

all future attended → 30 classes.

Already at or above target → 0 classes.

Scenario B searches the smallest future count that reaches the target.



9. Sample Output

Output:

Current attendance: 66.67 %

Future classes → all are attended: 30

Future classes in 4-out-of-5 scenario: 45

10. Conclusion

The program uses attendance percentages and loops to calculate the minimum recovery classes for two future attendance patterns.

CASE STUDY 6: STUDENT PERFORMANCE TREND ANALYSIS

1. Problem Statement

Compare marks across examinations, calculate improvements and declines, identify weak subjects, and classify the overall trend.

2. Objectives

- Apply the required rules to the supplied data.
- Use beginner-level Python constructs from the assigned syllabus.
- Display a clear and understandable result.

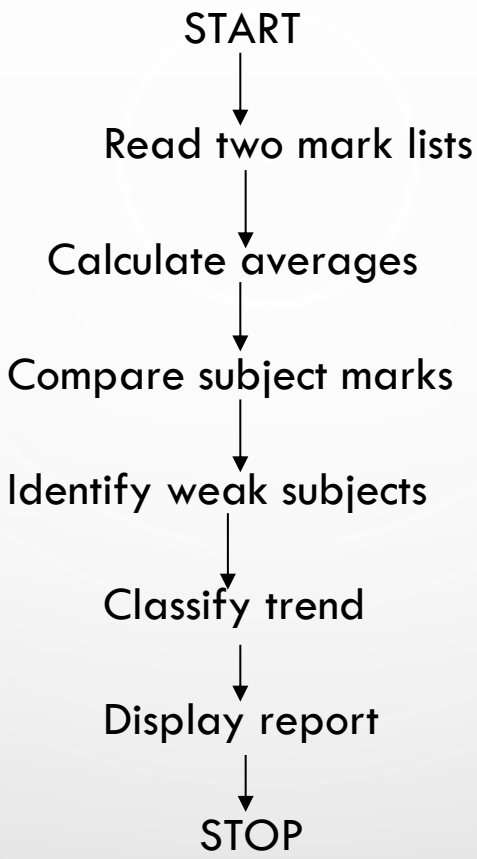
3. Assumptions

- Subjects: Python, Mathematics, English; marks are out of 100.
- A subject is weak if latest marks are below 50. Overall trend depends on change in average.

4. Algorithm

1. Store previous and latest marks in lists.
2. Calculate both averages.
3. Compare each subject's marks.
4. Identify latest marks below 50.
5. Classify overall average trend.

5. Flowchart



6. Pseudocode

```
FOR each subject
        change = latest - previous
        DISPLAY improved, declined, or unchanged
        IF latest < 50, add subject to weak list
CALCULATE previous and latest averages
IF latest average > previous average, trend = Improving
ELSE IF equal, trend = Unchanged
```

ELSE trend = Declining
DISPLAY report
STOP

7. Python Source Code

```
subjects = ["Python", "Mathematics", "English"]
previous = [65, 48, 70]
latest = [78, 45, 75]
weak = [ ]
for i in range(len(subjects)):
    change = latest[i] - previous[i]
    if change > 0:
        status = "Improved"
    elif change < 0:
        status = "Declined"
    else:
        status = "No change"
    print(subjects[i], status, "by", abs(change))
    if latest[i] < 50:
        weak.append(subjects[i])
old_avg = sum(previous) / len(previous)
new_avg = sum(latest) / len(latest)
print("Previous average:", round(old_avg, 2))
print("Latest average:", round(new_avg, 2))
if new_avg > old_avg:
```

```
    print("Overall trend: Improving")
elif new_avg < old_avg:
    print("Overall trend: Declining")
else:
    print("Overall trend: Unchanged")
print("Weak subjects:", weak)
```

8. Test Cases

Tests:

[65,48,70] ⟶ [78,45,75] gives two improvements,
one decline,
Mathematics weak,
overall improving.
Equal lists ⟶ unchanged.
Lower latest average fi declining.

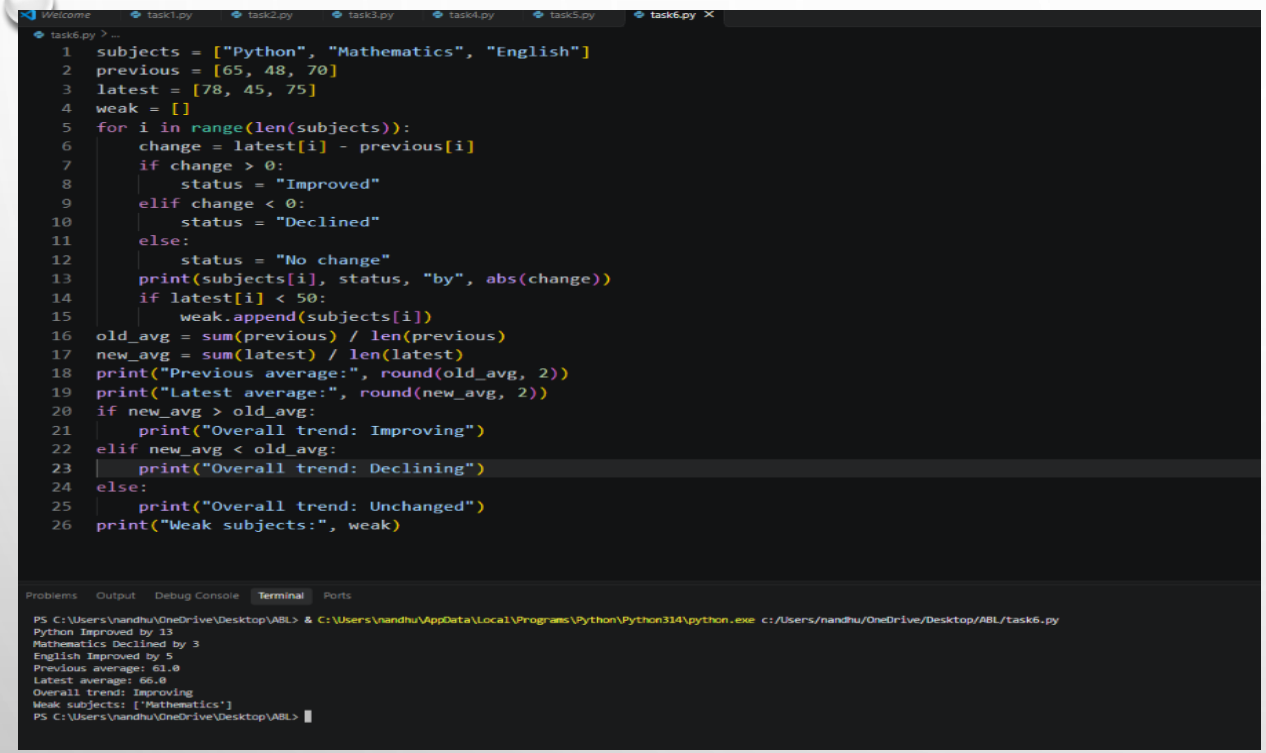
9. Sample Output

Output:

```
Python Improved by 13
Mathematics Declined by 3
English Improved by 5
Previous average: 61.0
Latest average: 66.0
Overall trend: Improving Weak subjects: ['Mathematics']
```

10. Conclusion

The program identifies subject-wise changes and overall performance trends using lists, loops, conditions, and arithmetic



```python
subjects = ["Python", "Mathematics", "English"]
previous = [65, 48, 70]
latest = [78, 45, 75]
weak = []
for i in range(len(subjects)):
    change = latest[i] - previous[i]
    if change > 0:
        status = "Improved"
    elif change < 0:
        status = "Declined"
    else:
        status = "No change"
    print(subjects[i], status, "by", abs(change))
    if latest[i] < 50:
        weak.append(subjects[i])
old_avg = sum(previous) / len(previous)
new_avg = sum(latest) / len(latest)
print("Previous average:", round(old_avg, 2))
print("Latest average:", round(new_avg, 2))
if new_avg > old_avg:
    print("Overall trend: Improving")
elif new_avg < old_avg:
    print("Overall trend: Declining")
else:
    print("Overall trend: Unchanged")
print("Weak subjects:", weak)
```

```
PS C:\Users\nandhu\OneDrive\Desktop\ABL> & C:\Users\nandhu\AppData\Local\Programs\Python\Python314\python.exe c:/Users/nandhu/OneDrive/Desktop/ABL/task6.py
Python Improved by 13
Mathematics Declined by 3
English Improved by 5
Previous average: 61.0
Latest average: 66.0
Overall trend: Improving
Weak subjects: ['Mathematics']
PS C:\Users\nandhu\OneDrive\Desktop\ABL>
```

CASE STUDY 7: STUDENT TIMETABLE CONFLICT DETECTION

1. Problem Statement

Compare course schedules stored as tuples and detect overlapping classes on the same day.

2. Objectives

- Apply the required rules to the supplied data.
- Use beginner-level Python constructs from the assigned syllabus.
- Display a clear and understandable result.

3. Assumptions

- Each tuple stores (course, day, start hour, end hour, room).
- Times use 24-hour whole numbers. End time equal to another start time is not a clash.

4. Algorithm

1. Represent schedules as tuples in a list.
2. Compare each pair using nested loops.
3. Check same day and overlapping intervals.
4. Display every clash and timetable validity.

5. Flowchart


START
Store timetable tuples
Compare each pair
Same day?
Overlap?
Report clash
Continue
Display validity
STOP

6. Pseudocode

```
FOR i from 0 to number of classes - 1
    FOR j from i + 1 to number of classes - 1
        unpack both tuples
        IF day1 == day2 AND start1 < end2 AND start2 < end1
            DISPLAY conflict
            conflicts = conflicts + 1
IF conflicts == 0, DISPLAY valid
ELSE DISPLAY invalid
STOP
```

7. Python Source Code

```
timetable = [
    ("Python", "Monday", 9, 10, "R101"),
    ("Maths", "Monday", 9, 11, "R102"),
    ("English", "Monday", 11, 12, "R103"),
    ("Physics", "Tuesday", 10, 11, "R101")
]
conflicts = 0
for i in range(len(timetable)):
    for j in range(i + 1, len(timetable)):
        course1, day1, start1, end1, room1 = timetable[i]
        course2, day2, start2, end2, room2 = timetable[j]
        if day1 == day2 and start1 < end2 and start2 < end1:
            print("Conflict:", course1, "and", course2, "on", day1)
```

```
            conflicts += 1
if conflicts == 0:
    print("Timetable is valid")
else:
    print("Total conflicts:", conflicts)
    print("Timetable is not valid")
```

8. Test Cases

Tests:

Monday 9–10 and Monday 9–11 ⟶ conflict.

Monday 9–10 and Monday 10–11 ⟶ no conflict.

Monday and Tuesday at same time ⟶ no conflict.

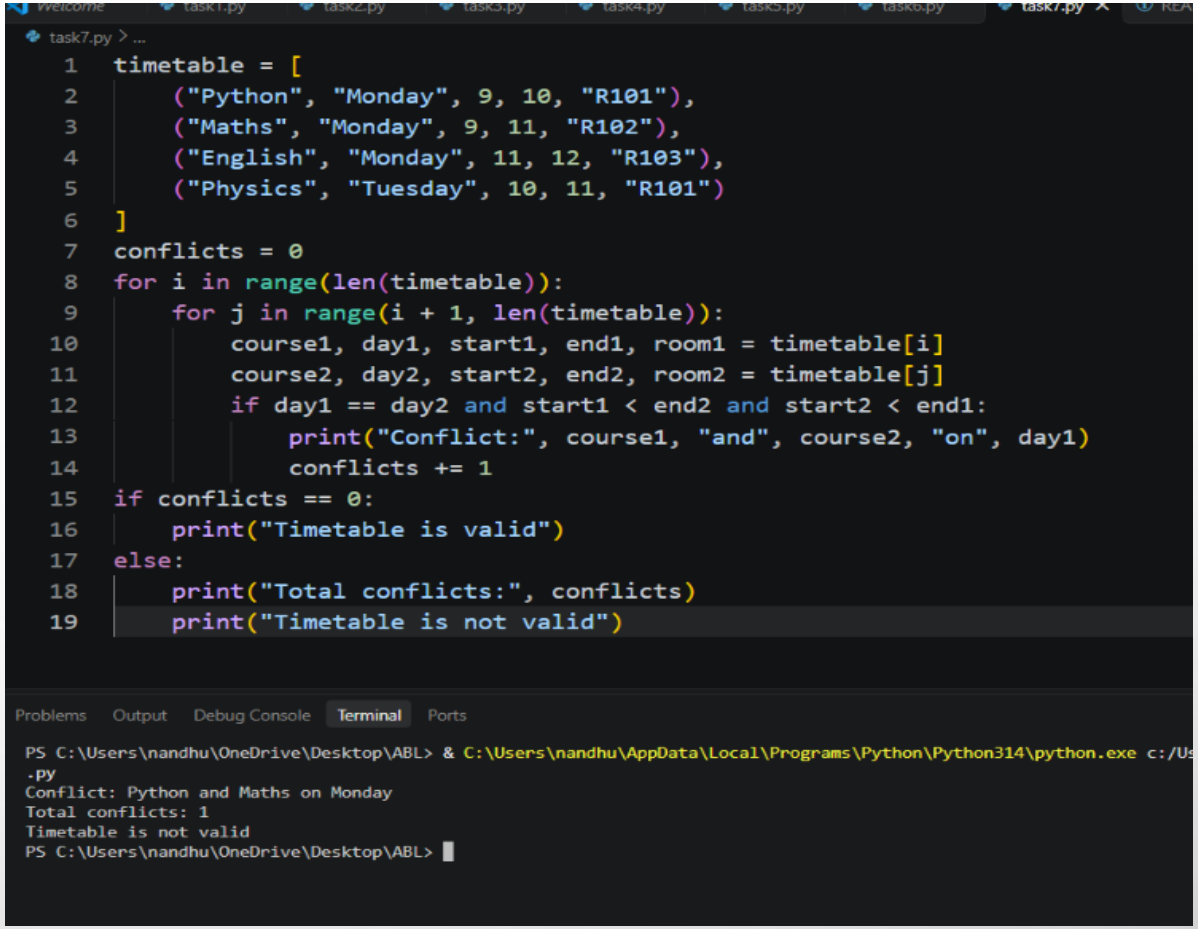
9. Sample Output

Output:

Conflict: Python and Maths on Monday

Total conflicts: 1

Timetable is not valid



10. Conclusion

The program detects overlapping class periods using tuples, tuple unpacking, nested loops, and conditions.

CASE STUDY 8: PASSWORD POLICY VALIDATION ENGINE

1. Problem Statement

Validate password strings against length, character-category, repetition, and prohibited-substring rules, explaining every failed condition.

2. Objectives

- Apply the required rules to the supplied data.
- Use beginner-level Python constructs from the assigned syllabus.
- Display a clear and understandable result.

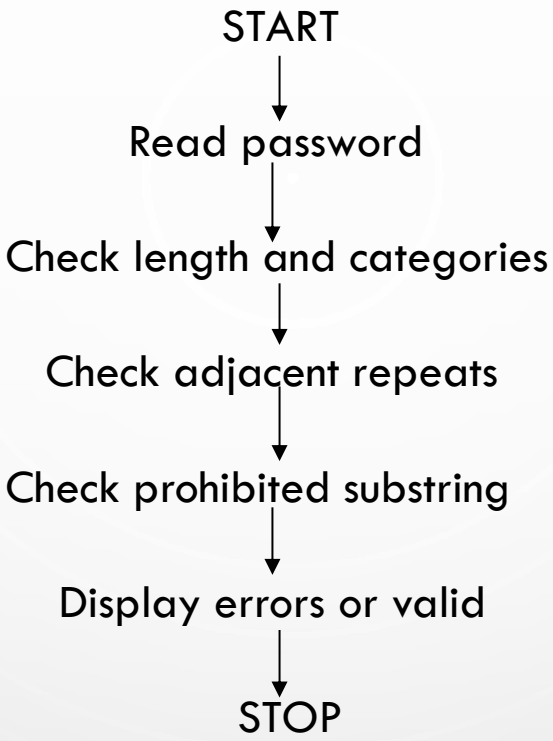
3. Assumptions

- Minimum length 8; must include uppercase, lowercase, digit, and one special character from !@#$%^&*.
- Adjacent repeated characters are not allowed. The substring 'password' is prohibited, ignoring case.

4. Algorithm

1. Read a password.
2. Check minimum length.
3. Loop through characters and set category flags.
4. Check adjacent repeated characters.
5. Check prohibited substring.
6. Display valid status or every error.

5. Flowchart



6. Pseudocode

```
INPUT password
errors = empty list
CHECK length >= 8
SET category flags False
FOR each character
    identify uppercase, lowercase, digit, special
CHECK adjacent characters for repetition
IF 'password' in lowercase password, add error
IF errors empty, DISPLAY valid
```

ELSE DISPLAY all errors
STOP

7. Python Source Code

```
password = "PassWord11"
errors = []
if len(password) < 8:
    errors.append("Use at least 8 characters")
upper = lower = digit = special = False
allowed_special = "!@#$%^&*"
for ch in password:
    if ch >= "A" and ch <= "Z":
        upper = True
    elif ch >= "a" and ch <= "z":
        lower = True
    elif ch >= "0" and ch <= "9":
        digit = True
    elif ch in allowed_special:
        special = True
if not upper:
    errors.append("Add an uppercase letter")
if not lower:
    errors.append("Add a lowercase letter")
if not digit:
    errors.append("Add a digit“)
```

```
if not special:
    errors.append("Add a special character")
for i in range(len(password) - 1):
    if password[i] == password[i + 1]:
        errors.append("No consecutive repeated characters")
        break
if "password" in password.lower():
    errors.append("The word password is prohibited")
if len(errors) == 0:
    print("Password is valid")
else:
   print("Password is invalid")
       for error in errors:
          print("-", error)
```

8. Test Cases

Tests: ⟶

Abc@1234 ⟶ valid.

PassWord11 ⟶ no special character and prohibited word.

Aaa@1234 adjacent repeated characters.

9. Sample Output

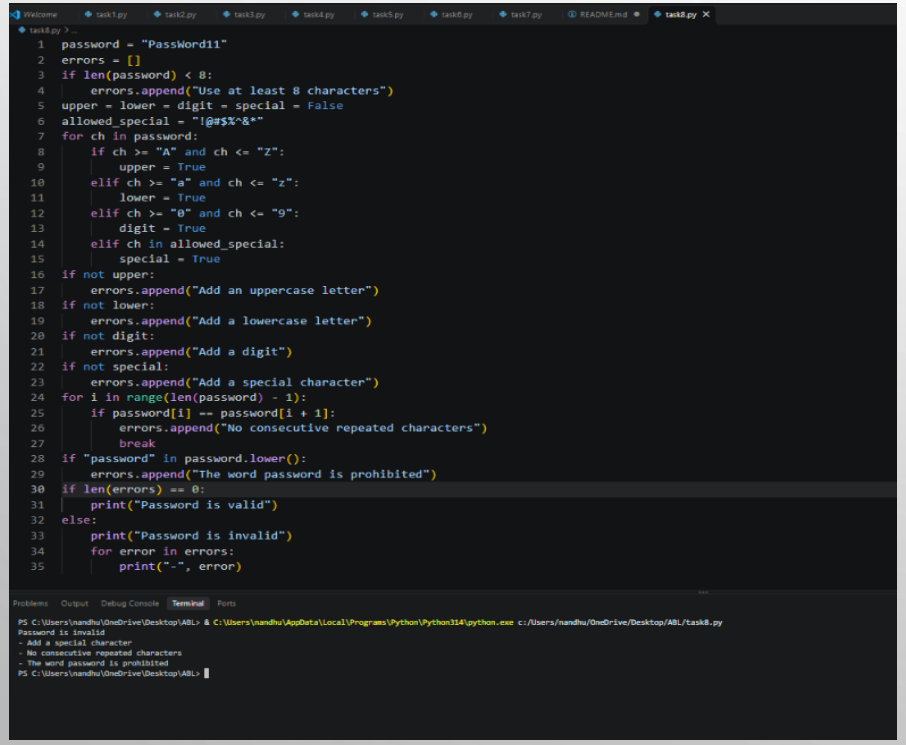
Output:

Password is invalid

- Add a special character
- - The word password is prohibited

10. Conclusion

The program checks several string rules and reports all validation errors using loops, lists, and conditions



CASE STUDY 9: STUDENT MARKS MATRIX ANALYSIS

1. Problem Statement

Analyse a matrix of student marks, calculate row-wise and column-wise statistics, identify the top performer, and find subjects with low averages.

2. Objectives

- Apply the required rules to the supplied data.
- Use beginner-level Python constructs from the assigned syllabus.
- Display a clear and understandable result.

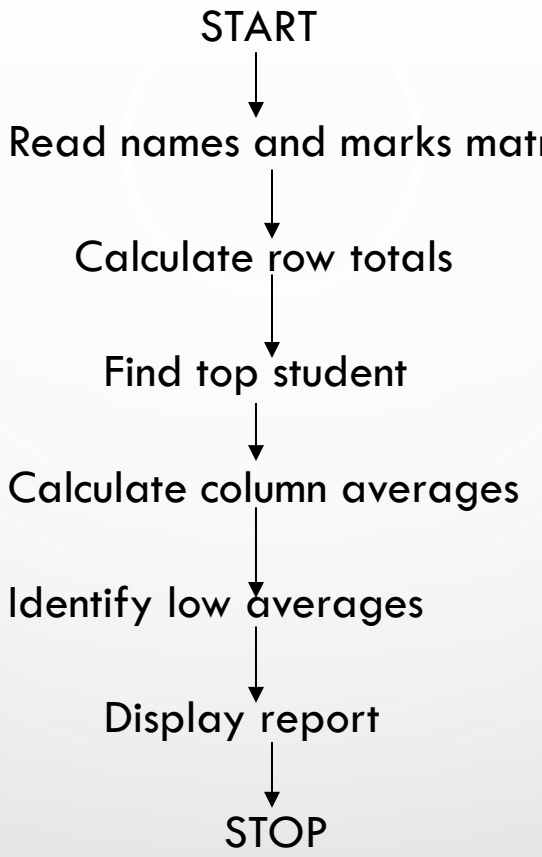
3. Assumptions

- Rows represent students and columns represent subjects.
- Marks are out of 100; a subject average below 50 is considered low.

4. Algorithm

1. Calculate each student's total and average.
2. Find the highest student total.
3. Calculate each subject's average by column.
4. Identify subjects below the low-average threshold.
5. Display the report.

5. Flowchart



6. Pseudocode

```
FOR each student row
    calculate total and average
FIND student with maximum total
FOR each subject column
    add marks from every student
    calculate column average
    IF average < 50, add subject to low list
```

DISPLAY student statistics, top student, subject averages
STOP

7. Python Source Code

```
students = ["Anu", "Balu", "Chitra"]
subjects = ["Python", "Maths", "English"]
marks = [[85, 78, 90], [70, 65, 72], [45, 40, 55]]
totals = []
for i in range(len(students)):
    total = sum(marks[i])
    totals.append(total)
    print(students[i], "Total:", total,
          "Average:", round(total / len(subjects), 2))
top = totals.index(max(totals))
print("Top performer:", students[top])
low = []
for j in range(len(subjects)):
    column_total = 0
    for i in range(len(students)):
        column_total += marks[i][j]
    average = column_total / len(students)
    print(subjects[j], "Average:", round(average, 2))
    if average < 50:
        low.append(subjects[j])
print("Low-average subjects:", low)
```

8. Test Cases

Tests:

Matrix [[85,78,90],[70,65,72],[45,40,55]] ⟶ Anu top performer.

If every Maths mark is 40 ⟶ Maths average 40 and is flagged.

Equal top totals ⟶ first matching student is shown.

9. Sample Output

Output:

Anu Total: 253 Average: 84.33

Balu Total: 207 Average: 69.0

Chitra Total: 140 Average: 46.67

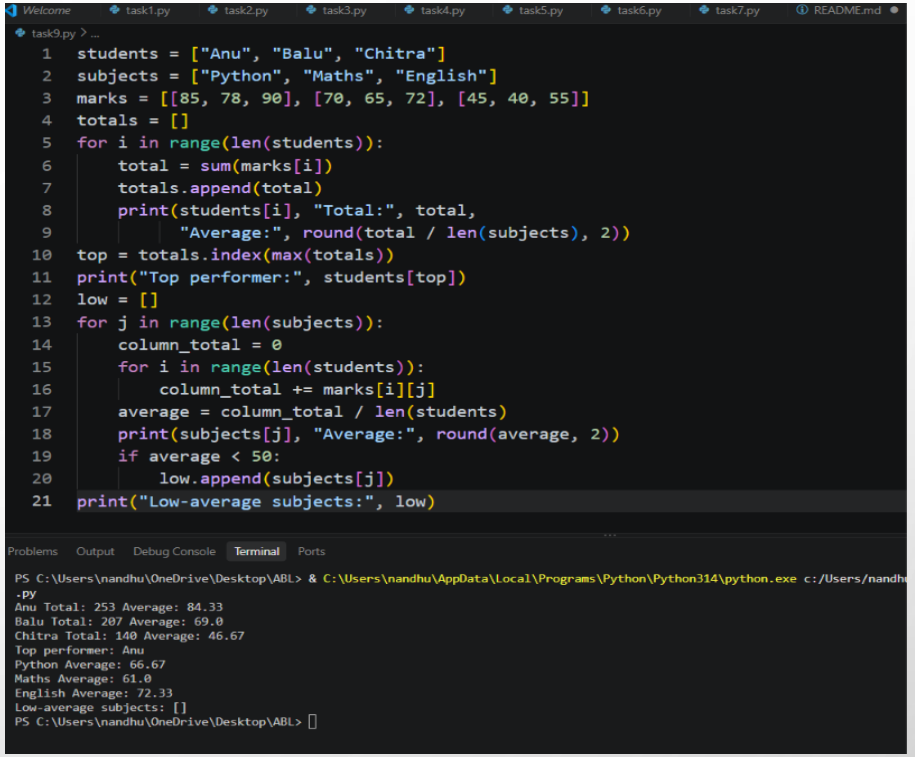
Top performer: Anu

Python Average: 66.67

Maths Average: 61.0

English Average: 72.33

Low-average subjects: [ ]



10. Conclusion

The program performs row-wise and column-wise analysis using nested lists, loops, indexing, and list operations.

CASE STUDY 10: SMART CANTEEN ORDER AND STOCK MANAGEMENT

1. Problem Statement

Process customer orders, calculate bills, update inventory, reject unavailable quantities, and summarize daily sales and remaining stock.

2. Objectives

- Apply the required rules to the supplied data.
- Use beginner-level Python constructs from the assigned syllabus.
- Display a clear and understandable result.

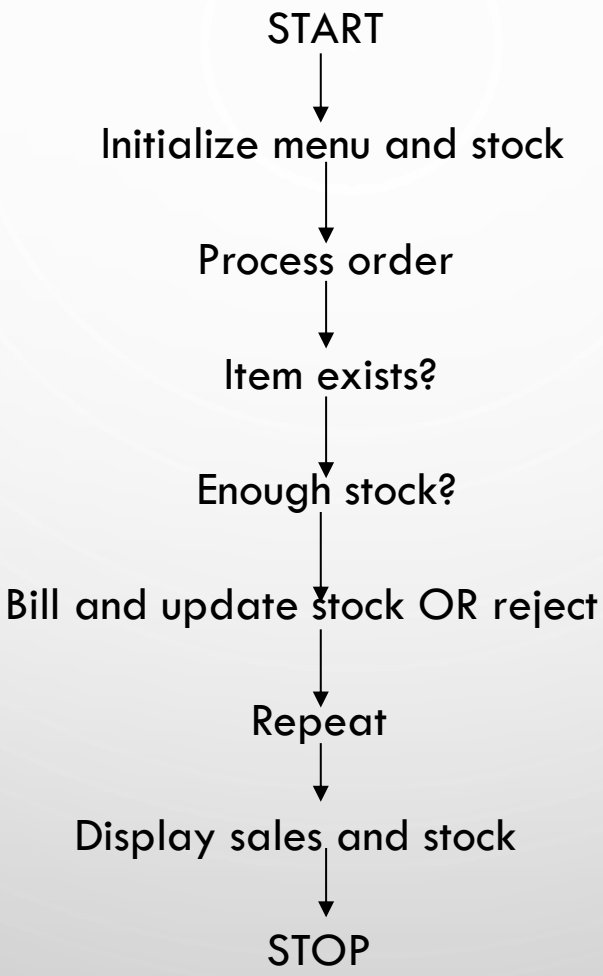
3. Assumptions

- Menu contains Idli, Dosa, and Juice with sample prices and stock.
- Orders are (item name, quantity). No tax or discount is applied.

4. Algorithm

1. Initialize menu, prices, and stock.
2. Process each order and find the item index.
3. Reject unknown items or quantities greater than stock.
4. For valid orders calculate bill, reduce stock, and add to sales.
5. Display daily sales and remaining stock.

5. Flowchart


START
Initialize menu and stock
Process order
Item exists?
Enough stock?
Bill and update stock OR reject
Repeat
Display sales and stock
STOP

6. Pseudocode

```
FOR each order (item, quantity)
    IF item not in menu, reject
    ELSE find item index
        IF quantity <= 0 OR quantity > stock[index], reject
        ELSE amount = price[index] * quantity
             reduce stock[index]
             add amount to sales
DISPLAY daily sales and remaining stock
STOP
```

7. Python Source Code

```
items = ["Idli", "Dosa", "Juice"]
prices = [30, 50, 25]
stock = [20, 15, 10]
orders = [("Idli", 4), ("Dosa", 3), ("Juice", 12), ("Pizza", 2)]
sales = 0
for item, quantity in orders:
    if item in items:
        i = items.index(item)
        if quantity <= 0:
            print("Invalid quantity:", item)
        elif quantity > stock[i]:
            print("Rejected:", item, "- insufficient stock")
        else:
```

```
            amount = prices[i] * quantity
            stock[i] -= quantity
            sales += amount
            print(item, "x", quantity, "Bill: Rs.", amount)
    else:
        print("Rejected - item unavailable:", item)
print("Daily sales: Rs.", sales)
print("Remaining stock:")
for i in range(len(items)):
    print(items[i], stock[i])
```

8. Test Cases

Tests:

Idli x 4 with stock 20 ⟶ accepted, Rs. 120.

Juice x 12 with stock 10 ⟶ rejected.

Pizza x 2 ⟶ unknown item, rejected

9. Sample Output

Output:

Idli x 4 Bill: Rs. 120

Dosa x 3 Bill: Rs. 150

Rejected: Juice - insufficient stock

Rejected - item unavailable: Pizza

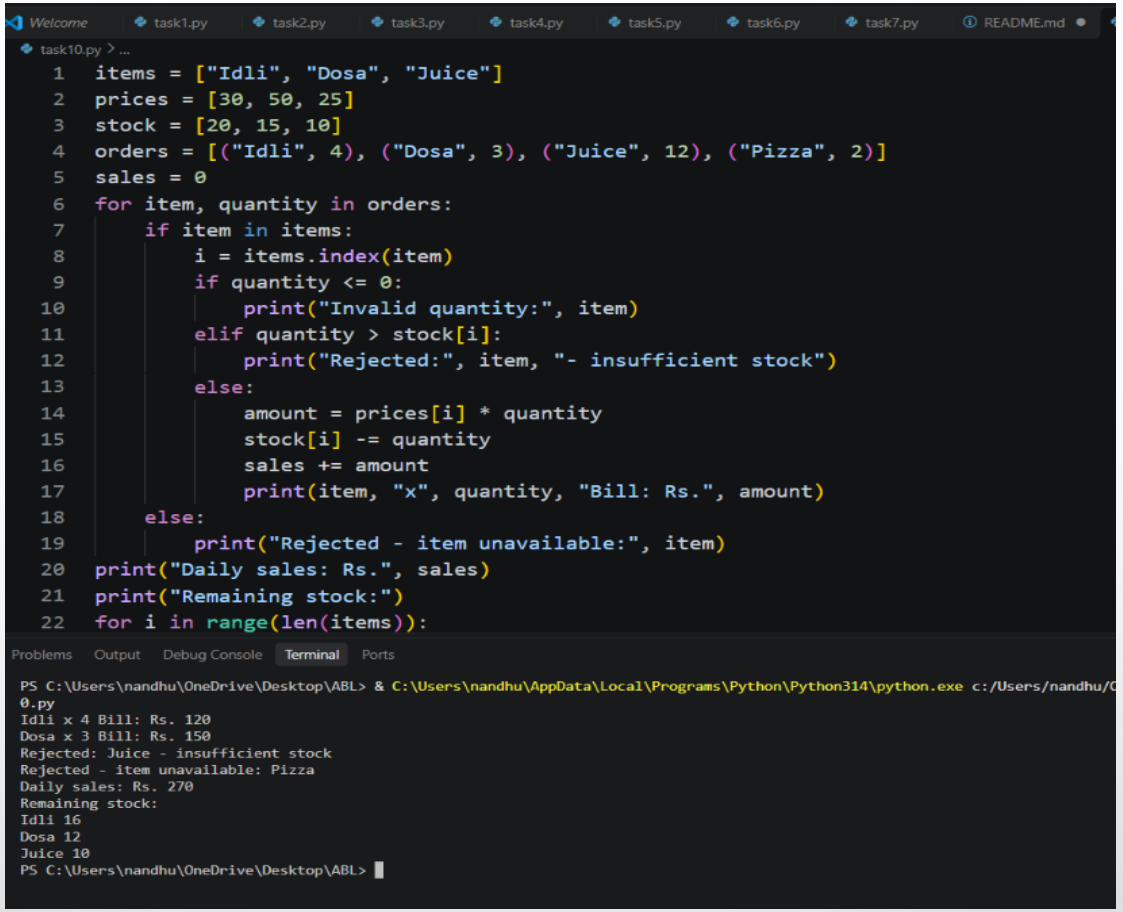
Daily sales: Rs. 270

Remaining stock:

Idli 16

Dosa 12

Juice 10



10. Conclusion

The program manages canteen orders and inventory using lists, tuples, loops, conditions, and indexing.